

# Practical guidelines for harness engineering with Claude Code.

Seven categories of harness components around the model,  
and how to build them in a real code base.

# Today

- |    |                                                                                                                             |         |
|----|-----------------------------------------------------------------------------------------------------------------------------|---------|
| 01 | What is harness engineering?<br><i>... and why we care</i>                                                                  | 03 - 05 |
| 02 | Seven categories of harness components<br> | 06 - 17 |
| 03 | Latest trends                                                                                                               | 18      |
| 04 | Demo                                                                                                                        | live    |

# Agent = Model + Harness

"Harness engineering" coined by **Mitchell Hashimoto** · Feb 2026  
Equation shaped by **Vivek Trivedy** at LangChain · shortly after

## *Why is this important?*

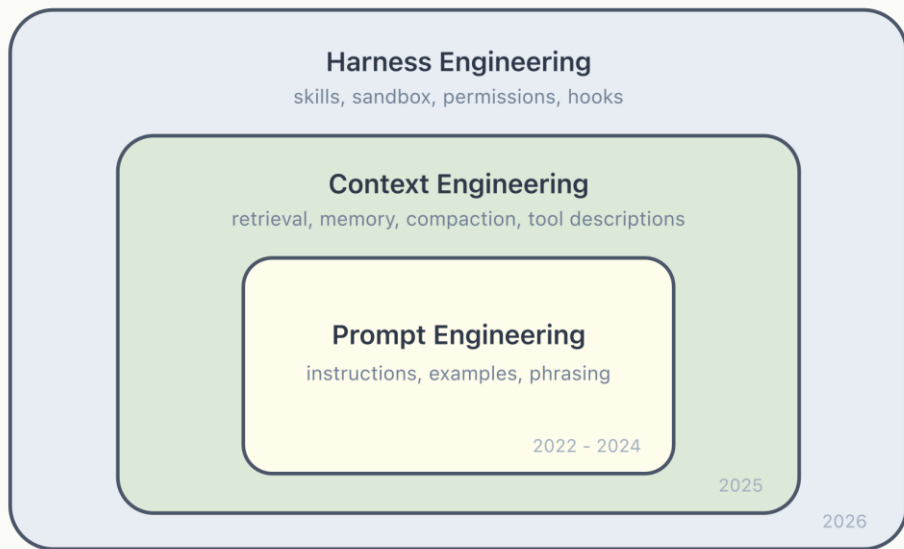
We are concerned about agentic code quality — and do not simply rely on prompting (vibe coding).

52.8% → 66.5% Terminal-Bench, harness only | 42% → 78% CORE-Bench, Opus in Claude Code

*Same model before and after — only the harness changed.*

# Three eras of engineering leverage

Each discipline subsumes the ones before it. You don't throw earlier work away — you embed it.



## WHY THE NESTING MATTERS

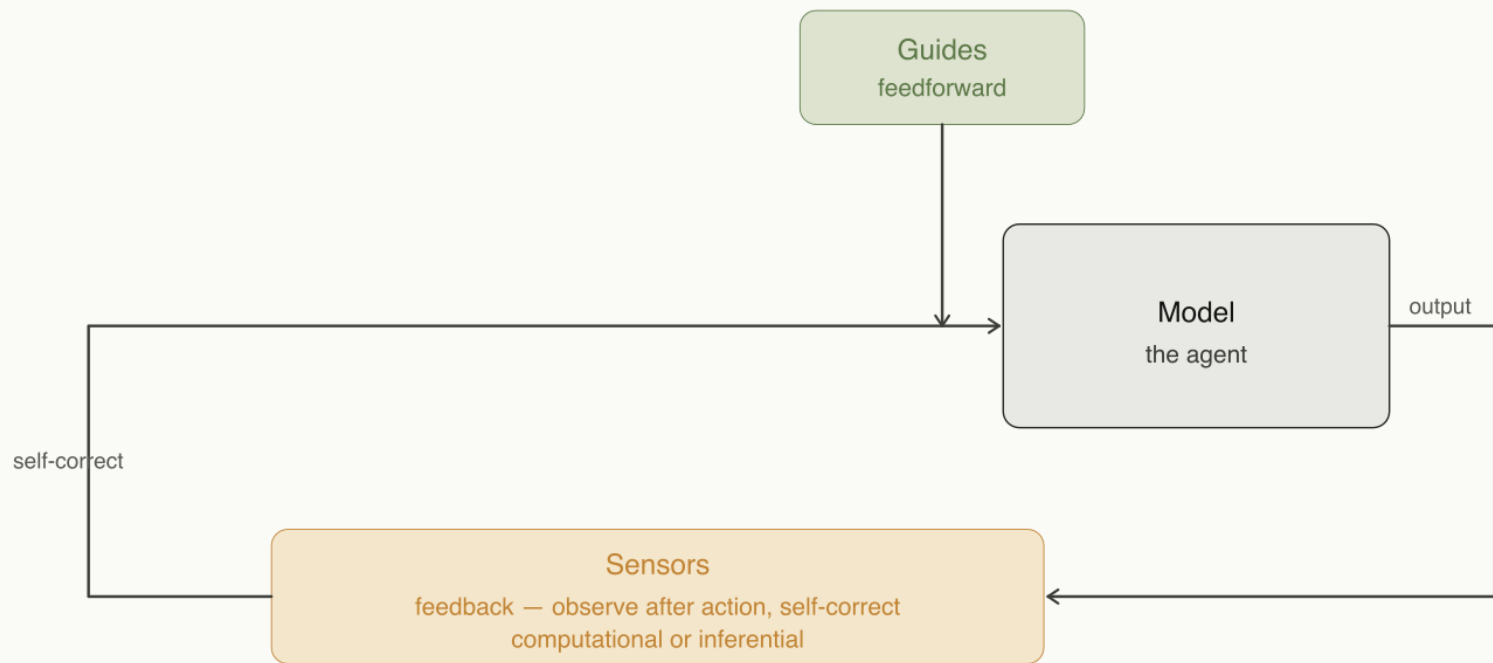
A good harness *contains* good context engineering.

Good context engineering *contains* good prompting.

Once the model is capable enough, the bottleneck migrates outward — to the entire execution environment around the loop.

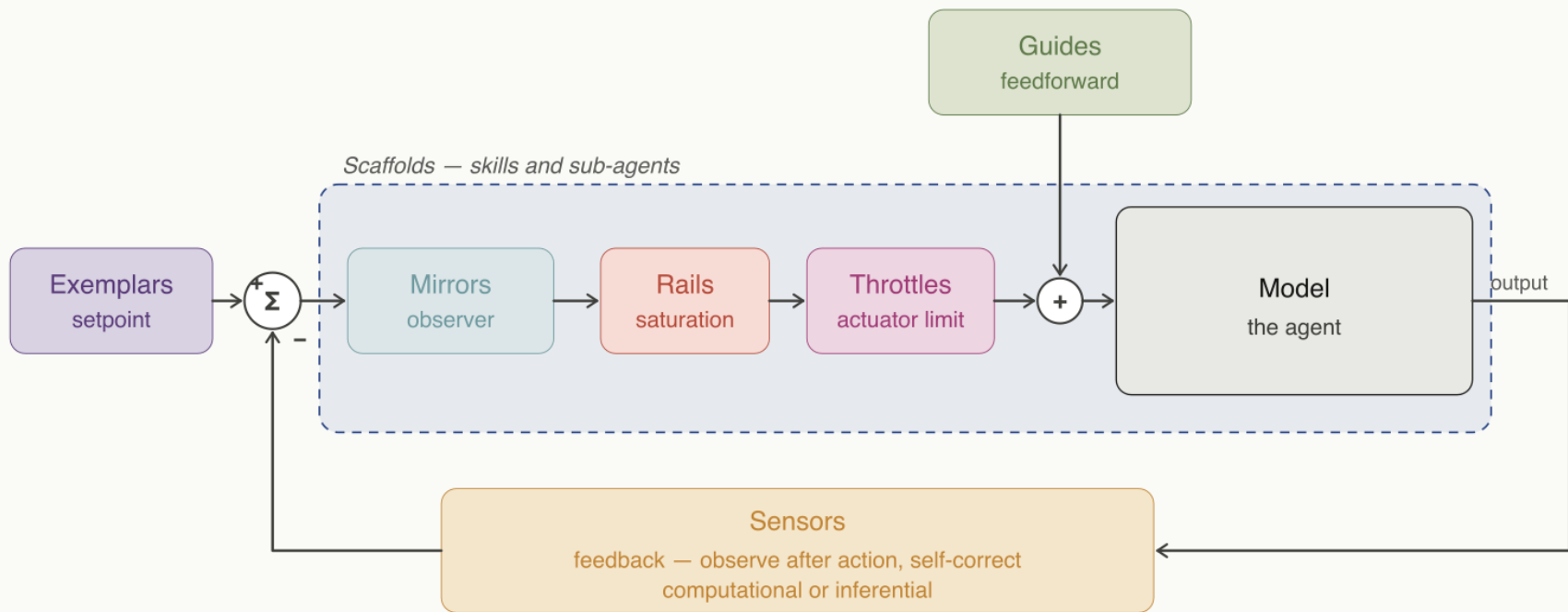
# Böckeler: Guides and Sensors

Feedforward (guides) and Feedback (sensors) are terms from Control Theory



# Seven categories of harness components

A framework for applied harness engineering inspired by control theory



# Guides

./CLAUDE.md — KEPT SHORT, SPECIFIC, FALSIFIABLE

## # Project conventions

- Stack: FastAPI + PostgreSQL 16.
- Tests: pytest, not unittest.
- HTTP clients: httpx, not requests.
- Branch off staging. Never commit to main.

## ### Commands

```
just test      # unit tests
just check     # ruff + ty + pytest
just migrate   # alembic upgrade head
```

## ### Iron laws

- Never git push --force to main.
- Never hand-edit alembic/versions/.

Bias what the agent produces  
*before* it acts.

A top-level CLAUDE.md, optional per-directory files for subsystems with their own conventions, and the system prompt itself.

### RULE OF THUMB

Keep it short — fewer than 200 lines.  
Hand-author your conventions.  
Prune ruthlessly.

# Sensors

## Observe what the agent did. Return an error signal.

Without sensors the agent runs open-loop — it cannot tell a fix from a regression.

### COMPUTATIONAL

Deterministic. The tool decides.

Linters, formatters, type checkers.

Test runners, build output.

Schema and contract validators.

### INFERENTIAL

Agent-interpreted. The model decides.

Screenshots — judge the layout.

Reviewer sub-agent — judge the diff.

Runtime logs — does the warning matter?

### RULE OF THUMB

Scope each sensor to the blast radius of the change.

Silent on success. Noisy on failure.

Emit failures as JSON — ideally in one fixed envelope shape.

# Sensors

## A type error becomes an error signal — no human in the middle.

1 — wire it up · `.claude/settings.json`

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/check.sh"
      }]
    }]
  }
}
```

2 — what comes back into the loop

```
{ "decision": "block",
  "reason": "ty: routes/invoices.py:42
    Argument of type 'str | None' cannot
    be assigned to parameter of type 'str'"
}
```

check.sh runs ruff and ty on the edited file only.

# Sensors

## No linter can tell you the invoice table stopped fitting.

1 – capture the evidence

```
# .claude/hooks/shot.sh (PostToolUse)
# matcher: Edit|Write on web/**

npx playwright screenshot \
  --viewport-size=1280,800 \
  http://localhost:5173/invoices \
  .cc/shot.png
```

The hook cannot judge a layout. It puts the pixels where the model can read them.

2 – hand it back with a question

```
{"hookSpecificOutput": {
  "hookEventName": "PostToolUse",
  "additionalContext":
    "UI changed. Read .cc/shot.png –
    does the invoice table still
    fit at 1280px?"}}
```

Narrow the question. “Does the table still fit?” gets an answer; “does this look good?” gets a compliment.

---

The hook returns no verdict — only evidence, and a question narrow enough to answer.

---

# Rails

DENY ENTRIES – THE ONLY LAYER WHERE "CANNOT" IS LITERAL

```
{
  "permissions": {
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force*)",
      "Bash(psql * DROP DATABASE*)",
      "Bash(DATABASE_URL=*prod* *)"
    ],
    "allow": [
      "Bash(just test*)",
      "Bash(just check*)"
    ]
  }
}
```

Invariants in CLAUDE.md are wishes.  
Invariants in permissions are laws.

## RULE OF THUMB

If an invariant matters, it goes here.

Not in the prose.

Not in a comment.

In the deny list.

# Scaffolds

skill – packaged procedure

```
# .claude/skills/release/  
---  
name: release  
description: Cut and tag  
             release after PR merge  
---  
1. just check  
2. bump pyproject.toml  
3. git tag && push --tags
```

## RULE OF THUMB

Reach for a skill when a procedure repeats.

# Scaffolds

skill – packaged procedure

```
# .claude/skills/release/  
---  
name: release  
description: Cut and tag  
           release after PR merge  
---  
1. just check  
2. bump pyproject.toml  
3. git tag && push --tags
```

sub-agent – critic

```
# .claude/agents/reviewer.md  
---  
name: reviewer  
description: Diff-only  
           critic, isolated context  
tools: Read, Grep,  
       Bash(git diff*)  
---  
Review the diff. Never open  
files outside it.
```

## RULE OF THUMB

Add a sub-agent when the main context must stay clean.

# Scaffolds

skill – packaged procedure

```
# .claude/skills/release/
---
name: release
description: Cut and tag
  release after PR merge
---
1. just check
2. bump pyproject.toml
3. git tag && push --tags
```

sub-agent – critic

```
# .claude/agents/reviewer.md
---
name: reviewer
description: Diff-only
  critic, isolated context
tools: Read, Grep,
  Bash(git diff*)
---
Review the diff. Never open
files outside it.
```

workflow – fan-out

```
# .claude/workflows/review.js
const DIRS =
  ["api", "web", "jobs"]

const bugs = await parallel(
  DIRS.map(d => () => agent(
    `Find bugs in ${d}`)))

const real = await parallel(
  bugs.map(b => () => agent(
    `Try to refute: ${b}`)))
return real.filter(Boolean)
```

## RULE OF THUMB

Add a workflow when the fan-out must be deterministic.

# Exemplars

Pointers, not prose

```
# CLAUDE.md – exemplars section
```

When adding a new HTTP endpoint:

- `routes/invoices.py` – req/resp shape
- `routes/test_invoices.py` – fixture style

When adding a new background job:

- `jobs/reminder_emails.py` – retry, idempotent

Not "write Pythonic Python." A pointer to a specific file as the canonical shape.

A worked example is worth a page of prose.

The agent pattern-matches to a trajectory instead of reasoning toward one. And exemplars are cheap — you're already writing the code.

## RULE OF THUMB

Keep the pointers fresh. A reference file the agent dutifully copies from long after the real convention has moved on is worse than no exemplar.

# Mirrors

Spend tokens on reflection before spending tokens on action.

**Plan mode** — a written plan before anything is changed.

**Extended thinking** — when the first answer is usually wrong.

**“Grill me” skill** — Matt Pocock's; it questions you first.

**Spec-driven development** — write the spec before the code.

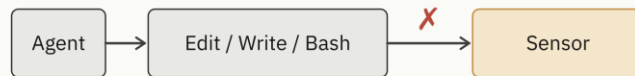
## RULE OF THUMB

Use a mirror when the failure mode is confident wrongness.

Not when information is missing.

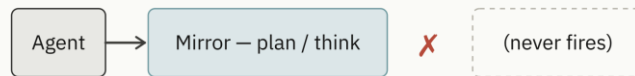
Skip mirrors for mechanical tasks.

### WITHOUT MIRROR



*caught **AFTER** the tool fires*

### WITH MIRROR



*caught **BEFORE** the tool fires*

Sensors catch wrongness *after* a tool call.

Mirrors catch it *before*.

# Throttles

## Deny the 3-minute build — and steer with the reason.

Agents rebuild after every edit; a cooldown forces batching. The same skeleton rate-limits any tool.

1 – the cooldown gate · PreToolUse on Bash

```
cmd=$(jq -r '.tool_input.command')
[[ "$cmd" == *"dotnet build"* ]] || exit 0
t=/tmp/build-throttle; now=$(date +%s)
last=$(cat "$t" 2>/dev/null || echo 0)
if (( now - last < 300 )); then
  jq -nc '{hookSpecificOutput: {
    hookEventName: "PreToolUse",
    permissionDecision: "deny",
    permissionDecisionReason: "..."} }' # → 2
  exit 0
fi; echo "$now" > "$t"
```

[full script](#) → [harness-engineering-demo](#) on GitHub

2 – the deny reason steers the model

- Bash(dotnet build FinTrack.sln)
  - └ Deny: "Build throttled: 212 s left.  
Keep editing and batch your changes;  
run one build afterwards."
- I'll make the remaining changes  
first, then build once.

### RULE OF THUMB

A rail says never; a throttle says not yet —  
and the reason tells the model what to do.

# Latest trends

Two buzzwords that dissolve into the seven categories — and two shifts in the harness itself.

the buzzword check

## Loop engineering

One name for **Sensors** (feedback), **Scaffolds** (topology) and **Throttles** (termination). It hides the hard part: each turn inherits the last one's context, so the loop is only as good as its **context engineering**.

---

## Graph engineering

Another name for process topology — which agent runs when, and on what. That is **Scaffolds** (workflows). (workflows are imperative scripts, not declarative graphs)

where the harness itself is heading

## Meta-harness

The harness tunes itself: an outer loop rewrites it, scores the result, and keeps what wins. It already beats hand-tuned harnesses (Lee et al., Mar 2026).

---

## Harness plugins

DeepSeek Harness (Aug 2026, MIT): models, tools, skills, sandboxes, loops, scheduling and the UI all mount as swappable plugins on the Cordis kernel.

# Stop wishing for a better model

Work on improving your harness setup

*Walk the seven categories. Where are you lopsided?*

# Thanks for listening

01   Guides

02   Sensors

03   Rails

04   Scaffolds

05   Exemplars

06   Mirrors

07   Throttles

All details — including extensive references — at:

[harness-engineering.carstenj.workers.dev](https://harness-engineering.carstenj.workers.dev)

Demo code available at:

[github.com/carsten-j/harness-engineering-demo](https://github.com/carsten-j/harness-engineering-demo)

Questions? Let's connect on LinkedIn:

[linkedin.com/in/carstenjoergensen](https://linkedin.com/in/carstenjoergensen)

# Further reading

**01** Harness engineering for coding agent users

Birgitta Böckeler · martinowler.com · Apr 2026

[martinowler.com/articles/harness-engineering.html](https://martinowler.com/articles/harness-engineering.html)

---

**02** Harness Engineering for Self-Improvement

Lilian Weng · Lil'Log · Jul 2026

[lilianweng.github.io/posts/2026-07-04-harness](https://lilianweng.github.io/posts/2026-07-04-harness)

---

**03** Agent Harness Engineering: A Survey

Tianyang Wang, Chandan K. Reddy et al. · 2026

[picrew.github.io/LLM-Harness](https://picrew.github.io/LLM-Harness)

---

**04** Meta-Harness: End-to-End Optimization of Model Harnesses

Yoonho Lee et al. · COLM 2026

[yoonholee.com/meta-harness](https://yoonholee.com/meta-harness)

---

**05** Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models

Qizheng Zhang et al. · ICLR 2026

[arxiv.org/abs/2510.04618](https://arxiv.org/abs/2510.04618)

---

**06** Model or Harness? An Interaction-Centric Taxonomy for Localizing Agent Failures

Harsh Raj et al. · arXiv · Jul 2026

[arxiv.org/abs/2607.28802](https://arxiv.org/abs/2607.28802)

---